

ASP.NET Web Forms and Ado.net

Introduction to ASP.NET Web Forms

ASP.NET Web Forms is a framework for building dynamic web applications. It uses a component-based approach, allowing developers to create user interfaces using drag-and-drop controls and write server-side code in languages like C#.

Module 2: Setting Up Your Development Environment

1. **Install Visual Studio:** Download and install Visual Studio Community, Professional, or Enterprise editions.
2. **Install SQL Server Management Studio (SSMS):** Download and install SSMS to manage SQL Server databases.
3. **Create a New ASP.NET Web Forms Project:** In Visual Studio, create a new "ASP.NET Web Application (.NET Framework)" project.

Module 3: Database Setup and ADO.NET Basics

Installing Microsoft SQL Server:

- If you're using SQL Server Express LocalDB, the instance name is typically (localdb)\MSSQLLocalDB.
- If you installed the full SQL Server (Developer, Standard, or Enterprise), the default instance name is often (local) or a dot (.).

Creating a Database and Table:

- Open SSMS and connect to your SQL Server instance.
- Create a new database (e.g., MyDatabase).
- Create a Users table with the following SQL:

```
SQL
USE MyDatabase;
GO
CREATE TABLE Users (
    UserID INT PRIMARY KEY IDENTITY(1, 1),
    Username VARCHAR(50) UNIQUE NOT NULL,
    Password VARCHAR(255) NOT NULL,
    Email VARCHAR(100),
    CreationDate DATETIME DEFAULT GETDATE()
);
GO
```

- Insert sample data into the Users table.

ADO.NET Introduction:

- ADO.NET (ActiveX Data Objects .NET) is a set of classes in the .NET Framework that provides access to data sources.

- **Core ADO.NET Components:**

- **SqlConnection:**

- Usage: Represents a connection to a SQL Server database.
 - Purpose: Establishes a communication channel between the application and the database.
 - The connection string is used when the object is created.
 - The Open() and Close() methods are used to manage the connection. ■

- **SqlCommand:**

- Usage: Represents a SQL query or stored procedure to be executed against the database.
 - Purpose: Allows you to send commands to the database for data retrieval, insertion, updating, or deletion.
 - The command text (SQL query) and the connection object are used when the object is created.
 - Parameters are used to prevent SQL injection.
 - The ExecuteReader(), ExecuteNonQuery(), and ExecuteScalar() methods are used to execute the command.

- **SqlDataReader:**

- Usage: Provides a forward-only, read-only stream of data from a SQL Server database.
 - Purpose: Efficiently reads data from the database row by row. ■ It is returned by the ExecuteReader() method of the SqlCommand object.
 - The Read() method is used to move to the next row.

- **DataSet:**

- It is used to create a copy of database into a local machine

- **ConfigurationManager:**

- Usage: Provides access to configuration settings in the web.config file. ■
 - Purpose: Allows you to store and retrieve application settings, such as connection strings, without hardcoding them in the code.
 - The ConnectionStrings property is used to retrieve connection strings.

When working with **SQL databases** in **ASP.NET (C#)**, we use **three main methods** to execute SQL commands:

ExecuteScalar() → Returns a **single value** (used for counting, getting a single ID, etc.).

Used for: `SELECT COUNT(*)`, `MAX()`, `MIN()`, `SUM()`, or getting an **auto-generated ID** after an insert.

ExecuteNonQuery() → Used for **INSERT, UPDATE, DELETE** (returns the number of rows affected).

- **Used for:** `INSERT`, `UPDATE`, `DELETE`, and `CREATE TABLE`
- **Returns:** Number of rows affected (int)
- **Does NOT** return query results.

ExecuteReader() → Retrieves **multiple rows** (used for `SELECT` queries)

- **Returns:** `SqlDataReader` (which we loop through)
- **Best for reading large amounts of data efficiently.**

Sample Codes For CRUD operations

1. Count Operation

```
string sql = "SELECT COUNT(*) FROM Users";

using (SqlCommand command = new SqlCommand(sql, connection))

{

    int userCount = (int)command.ExecuteScalar(); // Retrieves a single integer value

    Console.WriteLine("Total Users: " + userCount);

}
```

2. Insert Operation

```
string sql = "INSERT INTO Users (Username, Email, PasswordHash) OUTPUT INSERTED.UserID  
VALUES (@Username, @Email, @Password)";

using (SqlCommand command = new SqlCommand(sql, connection))

{
```

```
command.Parameters.AddWithValue("@Username", "JohnDoe");

command.Parameters.AddWithValue("@Email", "john@example.com");

command.Parameters.AddWithValue("@Password", "123456");


int newUserId = (int)command.ExecuteScalar(); // Returns the new UserID

Console.WriteLine("New User ID: " + newUserId);

}
```

3. Update Operation

```
string sql = "UPDATE Users SET PasswordHash = @NewPassword WHERE Username = @Username"; using (SqlCommand command = new SqlCommand(sql, connection))

{

    command.Parameters.AddWithValue("@NewPassword", "newpassword123");

    command.Parameters.AddWithValue("@Username", "Alice");


    int rowsAected = command.ExecuteNonQuery();

    Console.WriteLine(rowsAected + " row(s) updated.");

}
```

4. Delete Operation

```
string sql = "DELETE FROM Users WHERE Username = @Username";

using (SqlCommand command = new SqlCommand(sql, connection))

{

    command.Parameters.AddWithValue("@Username", "Alice");

    int rowsAected = command.ExecuteNonQuery();

    Console.WriteLine(rowsAected + " row(s) deleted.");

}
```

Connection String Setup:

- Add a connection string to the web.config file within the <connectionStrings> section:

XML

```
<connectionStrings>
  <add name="MyDatabaseConnection" connectionString="Data
Source=(localdb)\MSSQLLocalDB;Initial Catalog=MyDatabase;Integrated Security=True;"
providerName="System.Data.SqlClient" />
</connectionStrings>
```

Module 4: Creating a Login Form and Database Connectivity

1. Create login.aspx:

- Add a new Web Form to your project named login.aspx.
- Add the necessary HTML and ASP.NET controls:

HTML

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="login.aspx.cs"
Inherits="SqlConnections.login" %>
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
  <title>Login</title>
  <link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
</head>
<body>
  <div class="container">
    <div class="row justify-content-center">
      <div class="col-md-6">
        <form id="loginForm" runat="server">
          <div class="form-group">
            <label for="usernameTextBox">Username:</label>
            <asp:TextBox ID="usernameTextBox" runat="server"
CssClass="form-control"></asp:TextBox>
          </div>
          <div class="form-group">
            <label for="passwordTextBox">Password:</label>
            <asp:TextBox ID="passwordTextBox" runat="server" TextMode="Password"
CssClass="form-control"></asp:TextBox>
          </div>
          <asp:Button ID="loginButton" runat="server" Text="Login" CssClass="btn
btn-primary" OnClick="loginButton_Click" />
          <asp:Label ID="messageLabel" runat="server" CssClass="mt-3"></asp:Label>
        </form>
      </div>
    </div>
  </div>
```

```

        </div>
    </div>
</div>
</body>
</html>

```

2. Create login.aspx.cs (Code-Behind):

```

C#
using System;
using System.Configuration;
using System.Data.SqlClient;

namespace SqlConnections
{
    public partial class login : System.Web.UI.Page
    {
        protected void loginBuon_Click(object sender, EventArgs e)
        {
            string username = usernameTextBox.Text;
            string password = passwordTextBox.Text;
            string connectionString =
                ConfigurationManager.ConnectionStrings["MyDatabaseConnection"].ConnectionString;

            using (SqlConnection connection = new SqlConnection(connectionString)) {
                string sql = "SELECT * FROM Users WHERE Username = @Username AND
                    Password = @Password";
                using (SqlCommand command = new SqlCommand(sql, connection))
                {
                    command.Parameters.AddWithValue("@Username", username);
                    command.Parameters.AddWithValue("@Password", password); // Important security
                    risk, see note.

                    try
                    {
                        connection.Open();
                        using (SqlDataReader reader = command.ExecuteReader())
                        {
                            if (reader.Read())
                            {
                                Response.Redirect("Success.aspx");
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

        else
        {
            messageLabel.Text = "Incorrect username or password.";
        }
    }
}
catch (Exception ex)
{
    messageLabel.Text = "Error: " + ex.Message;
}
}
}
}
}
}
}
}
}
}

```

3. Create Success.aspx:

- Add a new web form named Success.aspx. This will be the page users are redirected to after successful login.

Module 5: Data Binding in Web Forms

1. Introduction to Data Binding:

- Data binding establishes a connection between a data source and a UI control.

2. DataSource Controls:

- **SqlDataSource:**
 - Connects to a SQL Server database and retrieves data using SQL queries. ■
- Example:

HTML

```

<asp:SqlDataSource ID="SqlDataSource1" runat="server"
    ConnectionString="Data Source=YourServer;Initial Catalog=YourDatabase;Integrated
    Security=True"
    SelectCommand="SELECT * FROM Products">
</asp:SqlDataSource>
<asp:GridView ID="GridView1" runat="server" DataSourceID="SqlDataSource1"
    AutoGenerateColumns="true"></asp:GridView>

```

- **ObjectDataSource:**

- Connects to a business object and retrieves data by calling methods.

Introduction to ASP.NET AJAX

ASP.NET AJAX is a framework that allows developers to create rich, interactive web applications. It enables partial page updates, reducing the need for full-page reloads.

💡 Benefits of ASP.NET AJAX

- ✓ Faster Performance – Only updates parts of the page instead of reloading everything.
- ✓ Better User Experience – No page flickering or full refreshes.
- ✓ Reduced Server Load – Only necessary data is sent to and from the server.
- ✓ Enhanced UI – Interactive elements like auto-complete, animations, and pop-ups.

💡 How It Works

The client (browser) sends an asynchronous request to the server.

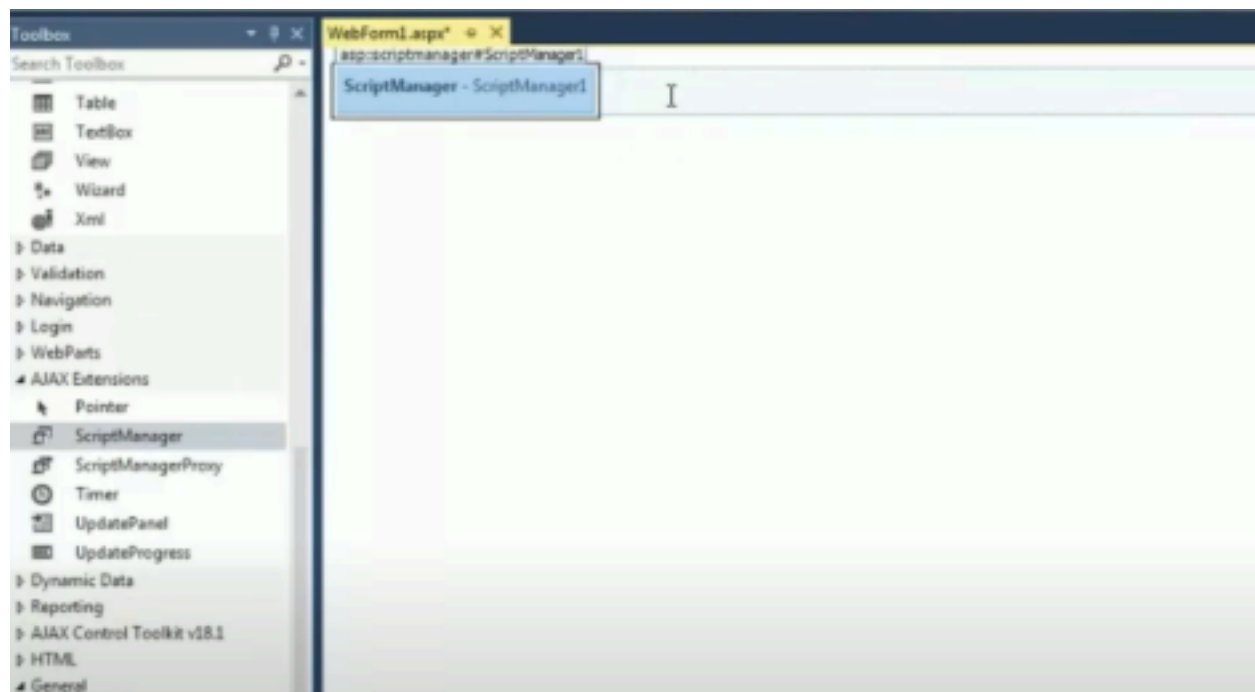
The server processes the request and sends back only the updated content.

The AJAX engine updates the page without refreshing it.

ScriptManager Control: This is required for managing and enabling AJAX functionality in a Web Forms page. It handles the loading of client-side scripts and facilitates AJAX operations.

It is used to manage script libraries, track the page request, and allow partial page updates.

It should be placed on the page inside the <form> tag.



source code:

UpdatePanel Control: This control is used to specify parts of the page that need to be updated asynchronously. When the page makes an AJAX request, only the content inside the `UpdatePanel` will be refreshed, rather than the entire page.

- It works in conjunction with the **ScriptManager** to provide AJAX-like functionality in ASP.NET Web Forms.
- The `UpdatePanel` is defined by setting its `UpdateMode` property (either `Always` or `Conditional`).
- Use script manager and within it use updatePanel controls and the controls

Ajax Buttons-Script Manager and Update Controls:

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="ajax.aspx.cs"
Inherits="CRUD.ajax" %>
```

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>AJAX Example</title>
</head>
<body>
    <form id="form1" runat="server">
        <!-- ScriptManager must be inside the form -->
        <asp:ScriptManager ID="ScriptManager1" runat="server" />

        <div class="container">
            <div class="form-group">
                <label for="usernameTextBox">Username:</label>
                <asp:TextBox ID="usernameTextBox" runat="server"
CssClass="form-control"></asp:TextBox>
            </div>

            <div class="form-group">
                <label for="passwordTextBox">Password:</label>
                <asp:TextBox ID="passwordTextBox" runat="server" TextMode="Password"
CssClass="form-control"></asp:TextBox>
            </div>

            <div class="form-group">
                <label for="newPasswordTextBox">New Password (for Update only):</label>
```

```

        <asp:TextBox ID="newPasswordTextBox" runat="server" TextMode="Password"
        CssClass="form-control"></asp:TextBox>
    </div>

    <!--  UpdatePanel should be separate from ScriptManager -->
    <asp:UpdatePanel runat="server">
        <ContentTemplate>
            <asp:Label ID="messageLabel" runat="server" CssClass="mt-3"></asp:Label>
            <br />
            <asp:Button ID="Delete" runat="server" Text="Delete Account" CssClass="btn
btn-danger" />
            <asp:Button ID="Update" runat="server" Text="Update" CssClass="btn
btn-primary" />
        </ContentTemplate>
    </asp:UpdatePanel>
</div>
</form>
</body>
</html>

```

The **AJAX Control Toolkit** is a collection of controls that enhance the AJAX functionality in ASP.NET Web Forms applications. These controls are designed to provide additional features such as animations, effects, and various interactive UI elements without writing much JavaScript code.

Some popular controls from the AJAX Control Toolkit include:

- **CalendarExtender**: Provides an enhanced calendar control with AJAX functionality.
- **ModalPopupExtender**: Displays a modal dialog (popup) on the page without needing to reload the page.
- **AutocompleteExtender**: Provides an auto-suggestion or autocomplete feature for text fields.

File Handling in ASP.NET

ASP.NET provides various classes to handle files on the server, such as reading and writing files, deleting files, or managing file directories. The **System.IO** namespace is primarily used for these operations.

Common File Handling Tasks in ASP.NET:

1. **Reading and Writing Files**: You can read and write to files using classes like `StreamReader` and `StreamWriter`, or using `FileStream`.

2. **Managing Directories:** You can create, delete, and list directories using `Directory` and `DirectoryInfo`.
3. **Managing Files:** You can copy, delete, move, and check if files exist using the `File` class.

FileHandling

`using System.IO;` → Enables file handling (reading, writing, deleting files).

1. Uploading a File:

```
protected void btnUpload_Click(object sender, EventArgs e)
{
    if (FileUpload1.HasFile)
    {
        string filePath = Server.MapPath("~/Uploads/") +
FileUpload1.FileName;
        FileUpload1.SaveAs(filePath);
        lblMessage.Text = "File uploaded successfully!";
        lblMessage.CssClass = "alert alert-success";
    }
    else
    {
        lblMessage.Text = "Please select a file to upload.";
        lblMessage.CssClass = "alert alert-warning";
    }
}
```

Description:

Checks if a file is selected (`FileUpload1.HasFile`). a.

Generates file path using `Server.MapPath("~/Uploads/")`:

b. Converts a **virtual path** (inside the project) to a **physical disk path**.

c. The file is stored inside the Uploads folder.

d. **Saves the file** to that location using `FileUpload1.SaveAs(filePath)`.

e. **Displays a success or warning message** in `lblMessage`.

2. Reading a File:

```
protected void btnRead_Click(object sender, EventArgs e)
```

```

{
    string filePath = Server.MapPath("~/Uploads/") +
FileUpload1.FileName;
    if (File.Exists(filePath))
    {
        lblContent.Text = File.ReadAllText(filePath);
        lblMessage.Text = "File content displayed.";
        lblMessage.CssClass = "alert alert-info";
    }
    else
    {
        lblMessage.Text = "File not found!";
        lblMessage.CssClass = "alert alert-danger";
    }
}

```

Description:

Checks if the file exists using **File.Exists(filePath)**.

Reads the content with **File.ReadAllText(filePath)** and displays it in **lblContent**.

Shows an appropriate message in **lblMessage**

3. Writing To a File:

Description:

1. **Server.MapPath("~/Uploads/")** → Converts a virtual path(~/Uploads/) to an absolute path on the server. 2.we use the **File.WriteAllText** method to **write content** to the file

4. Deleting a File

```

protected void btnDelete_Click(object sender, EventArgs e)
{
    string filePath = Server.MapPath("~/Uploads/") +
FileUpload1.FileName;
    if (File.Exists(filePath))
    {
        File.Delete(filePath);
        lblMessage.Text = "File deleted successfully!";
        lblMessage.CssClass = "alert alert-success";
        lblContent.Text = ""; // Clear file content
    }
    else

```

```
{  
    lblMessage.Text = "File does not exist!";  
    lblMessage.CssClass = "alert alert-danger";  
}
```

Description:

1. **Deletes the file** using `File.Delete(filePath)`. 2.

Clears the file content label (`lblContent.Text = ""`).